Reporte del Proyecto Final

Cliente HTML5 con MongoDB

Miguel Rodriguez

Abril 2026

Reporte del Proyecto Final

Asignatura: Programacion Web
Proyecto: Proyecto Final – Cliente OP
Estudiante: Miguel Rodriguez (1015-4714) y José Hidalgo (1015-5858)
Repositorio: Proyecto-Final-Web *(https://github.com/JoseHidalgoG/Proyecto-Final-Web)*

URL: jsea.me
Fecha: abril 2026

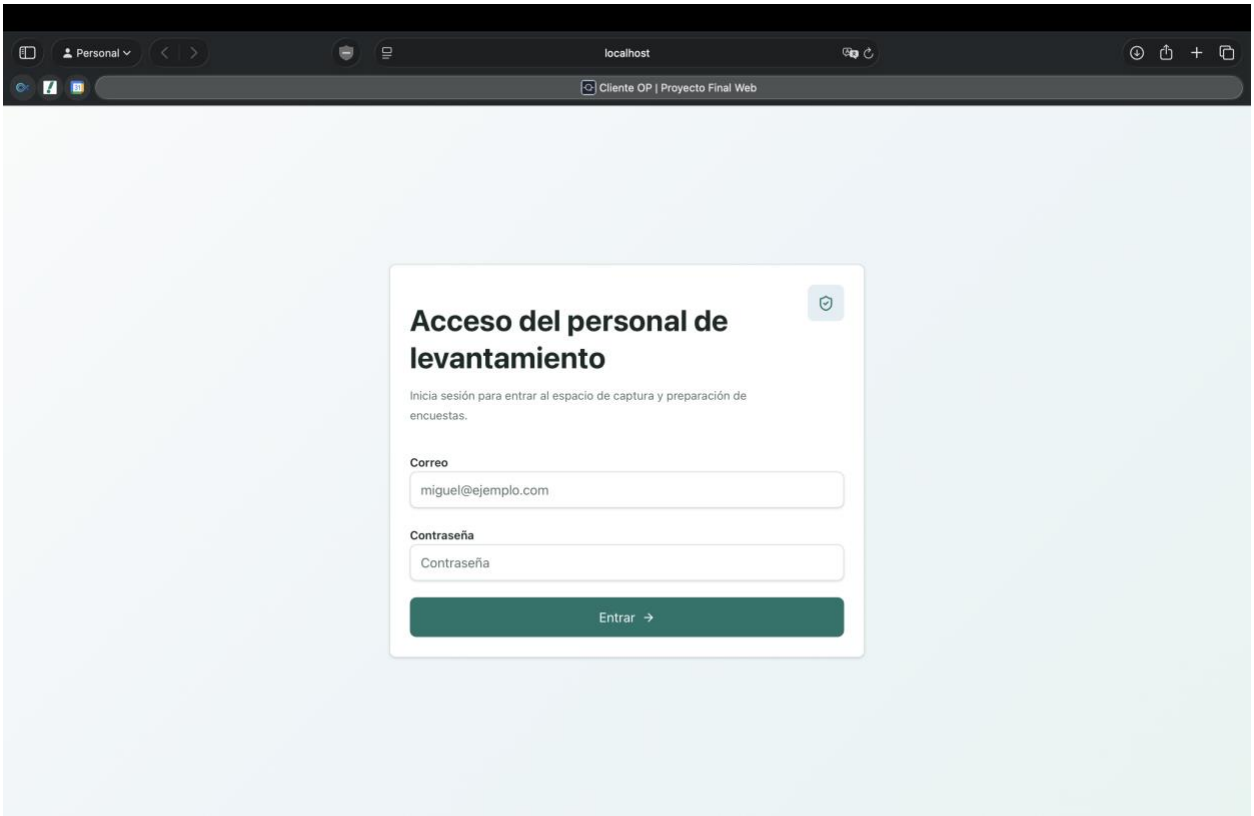
Esta es una aplicación web orientada a la captura de encuestas para la Oficina de Planeamiento de la PUCMM. El sistema fue construido con el objetivo de permitir que el personal de apoyo pueda registrar informacion desde dispositivos moviles, incluso cuando no exista conectividad, y sincronizar posteriormente los formularios con el servidor.

Aquí combinamos una interfaz web con un backend en Java basado en Javalin, almacenamiento persistente en MongoDB y mecanismos de comunicacion mediante REST, WebSocket y gRPC. El proyecto tambien incorpora almacenamiento local en el navegador para conservar formularios pendientes, captura de fotografia en Base64, geolocalizacion, autenticacion basada en JWT, administracion de usuarios y visualizacion geografica de los registros recibidos.

Aunque el requerimiento principal solicitaba una aplicacion web con funcionalidades avanzadas de HTML5, en este proyecto se decidio construir el frontend usando React. En mi caso, Miguel Rodriguez, ya tenia experiencia previa trabajando con React, por lo que esta tecnología me hizo el desarrollo mas comodo, limpio y organizado. React permitio separar la interfaz en componentes, manejar estados de forma controlada y estructurar las paginas de una manera mantenible. De igual forma, preferi utilizar Tailwind CSS frente a Bootstrap debido a mi experiencia previa con Tailwind y a la flexibilidad que ofrece para construir una interfaz responsive sin depender de componentes visuales demasiado rigidos.

Por otro lado, mi compañero de grupo, José Alberto Hidalgo se encargó del backend donde se manejó Javalin como microframework, como se dijo anteriormente.

El resultado es una aplicacion dividida en dos partes principales: un frontend encargado de la experiencia del usuario, la captura local y la sincronización, y un backend encargado de la seguridad, persistencia, servicios REST, canal WebSocket y servicio gRPC.



Objetivos del Proyecto

El proyecto busca cumplir con los siguientes objetivos solicitados:

- Construir una aplicación web usable desde computadoras, teléfonos inteligentes y tabletas.
- Utilizar Javalin como framework principal del servidor.
- Implementar almacenamiento de datos en MongoDB, usando un enfoque de persistencia organizado mediante modelos, repositorios y servicios.
- Permitir la captura de formularios con nombre, sector, nivel escolar, usuario registrador, foto y ubicación geográfica.
- Almacenar formularios localmente cuando el dispositivo no tenga conexión.
- Sincronizar formularios pendientes con el servidor mediante WebSocket y Web Workers.

- Proteger los servicios REST mediante autenticacion JWT manteniendo una separación y administración de usuarios.
- Mostrar en un mapa los formularios recibidos por el servidor.
- Publicar servicios REST y gRPC para operaciones relacionadas con formularios.
- Dockerizar la aplicación usando Docker.

Visión General de la Arquitectura

La arquitectura del proyecto sigue una separacion clara entre cliente y servidor. El frontend se ejecuta en el navegador y se encarga de la interaccion directa con el usuario. El backend expone los servicios necesarios para autenticar usuarios, guardar informacion, consultar formularios, administrar usuarios, sincronizar datos y atender clientes externos mediante REST y gRPC.

La aplicacion esta organizada alrededor de los siguientes bloques:

- **Frontend web:** construido con React en Vite, TypeScript y Tailwind CSS.
- **Backend Java:** construido con Javalin, servicios de negocio, repositorios y DTOs.
- **Persistencia:** MongoDB Atlas como base de datos NoSQL y Morphia como ODM.
- **Sincronizacion:** Web Workers en el navegador y WebSocket en el servidor.
- **Servicios externos:** API REST protegida con JWT y servicio gRPC.

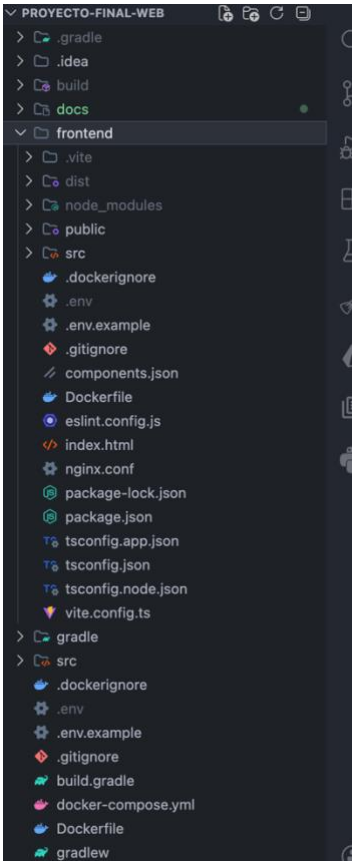
División entre Frontend y Backend

La división entre frontend y backend permite que cada parte del proyecto tenga responsabilidades bien definidas. El frontend se concentra en la experiencia del usuario, la captura de datos, el almacenamiento local y la presentacion visual. El backend se concentra en las reglas de negocio, la seguridad, la persistencia y la exposicion de servicios.

Esta separación tiene varias ventajas:

- Facilita el mantenimiento del codigo.
- Permite probar y desplegar cada parte de forma independiente.
- Reduce el acoplamiento entre la interfaz y la logica del servidor.
- Permite que otros clientes consuman el backend, como el cliente gRPC elaborado.
- Mejora la organizacion general del proyecto.

La estructura general del repositorio refleja esta separacion:



El directorio frontend contiene toda la aplicacion del navegador. El directorio src/main/java/org/example contiene el backend Java. El archivo docker-compose.yml une ambos servicios para facilitar el arranque del sistema completo.

Frontend

El frontend fue desarrollado con React con Vite, TypeScript y Tailwind CSS. Aunque React no era una exigencia real del proyecto, se eligio porque permite construir interfaces dinamicas de forma mas ordenada. En mi caso, Miguel Rodriguez, la experiencia previa con React hizo que el desarrollo fuera mas rapido, limpio y mantenible.

React ayudo a dividir la interfaz en paginas y componentes reutilizables. En lugar de manejar toda la aplicacion con archivos HTML y JavaScript sueltos, el proyecto usa modulos con responsabilidades específicas con sus tipos, componentes auxiliares, hooks, peticiones al backend y página principal como: autenticación, dashboard, captura de formularios, pendientes, mapa y administracion de usuarios.

Tambien se uso Tailwind CSS en lugar de Bootstrap, por la comodidad y experiencia previa con este ya que permite construir interfaces responsive directamente desde clases utilitarias sin tanto problema, permitiendo utilizar componentes prehechos que nos resten menos tiempo.

Autenticación en el Frontend

El usuario inicia sesion mediante la pantalla de login donde el frontend envía las credenciales al backend y, si son validas, el servidor devuelve la informacion del usuario y registra el token JWT en una cookie HTTP only. Así, este mantiene la sesión actual mediante un contexto de autenticacion, lo que permite proteger rutas privadas y mostrar opciones dependiendo del rol del usuario.

}

Captura de Formularios

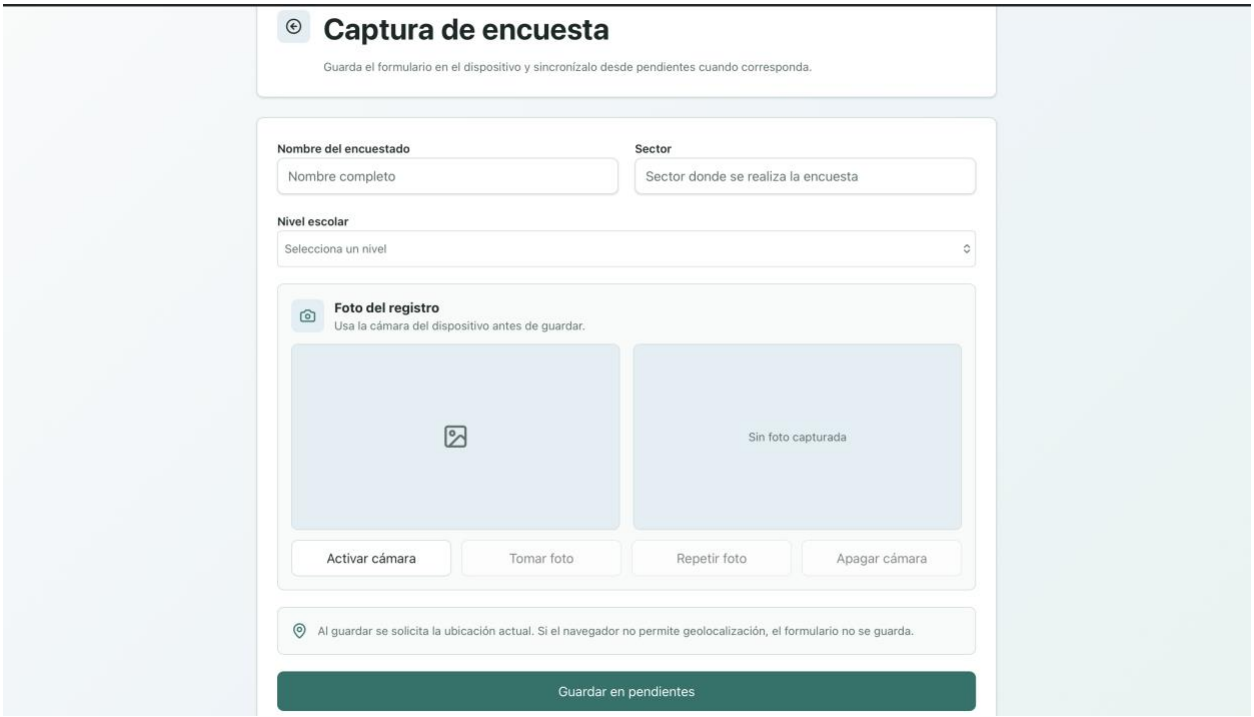
La pagina de captura permite registrar:

- Nombre del encuestado.

- Sector.
- Nivel escolar.
- Foto tomada desde la camara del dispositivo.
- Geolocalizacion actual.
- Usuario que realiza el registro.

La foto se obtiene desde la camara del dispositivo usando las capacidades del navegador, posteriormente se procesa mediante un canvas HTML y se convierte a Base64. Allí mismo la ubicacion se obtiene mediante la API de geolocalización del navegador y si la cámara o la ubicación no estan disponibles, la interfaz informa el problema y evita guardar formularios incompletos.

Una vez completado el formulario, la informacion no se envia directamente al servidor. Primero se guarda localmente en el IndexedDB del navegador como registro pendiente, permitiendo que el usuario continue trabajando incluso sin conexion.



Almacenamiento Local y Pendientes

Los formularios se guardan localmente en IndexedDB. Cada formulario local incluye los datos capturados, la foto en Base64, las coordenadas, el identificador del usuario, la fecha de creacion, la fecha de actualizacion y un estado local.

Los estados principales son:

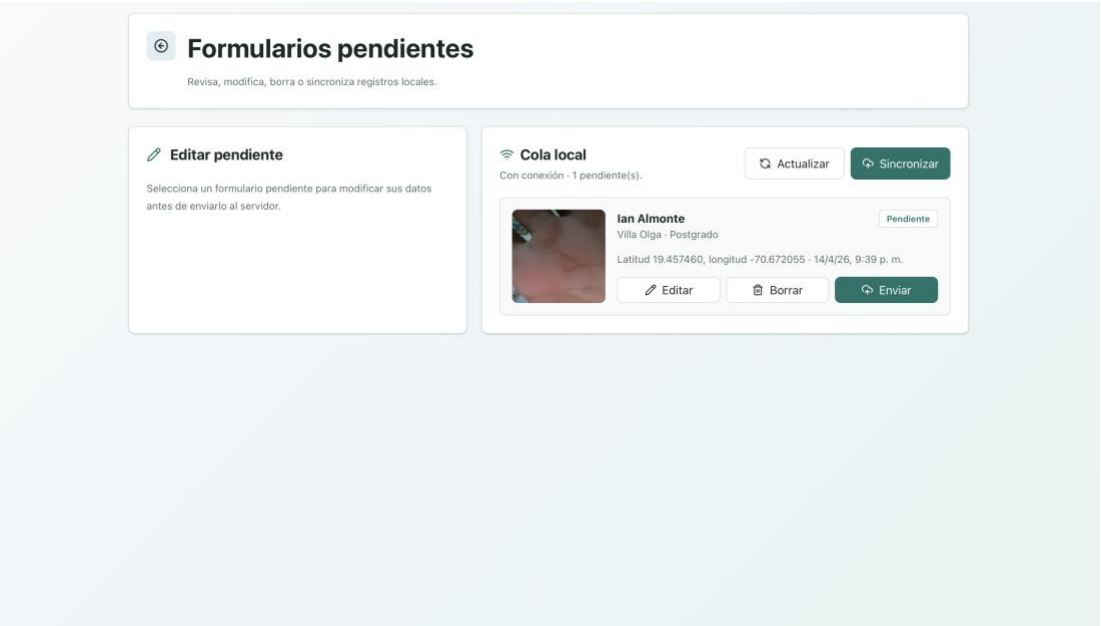
- PENDING: formulario pendiente de envio.
- ERROR: formulario que intento sincronizarse pero fallo.
- SYNCED: formulario sincronizado correctamente con el servidor.

La pantalla de pendientes permite revisar la cola local. Desde esta pantalla el usuario puede modificar o borrar un registro antes de enviarlo. Esto cumple con el requerimiento de permitir acciones sobre los formularios antes de su sincronizacion.

Sincronizacion con Web Workers y WebSocket

La sincronizacion se ejecuta desde un Web Worker. Esto evita que el proceso de envio bloquee la interfaz principal. El worker consulta los formularios pendientes en IndexedDB, valida que tengan foto y geolocalizacion, y luego abre una conexion WebSocket hacia el backend.

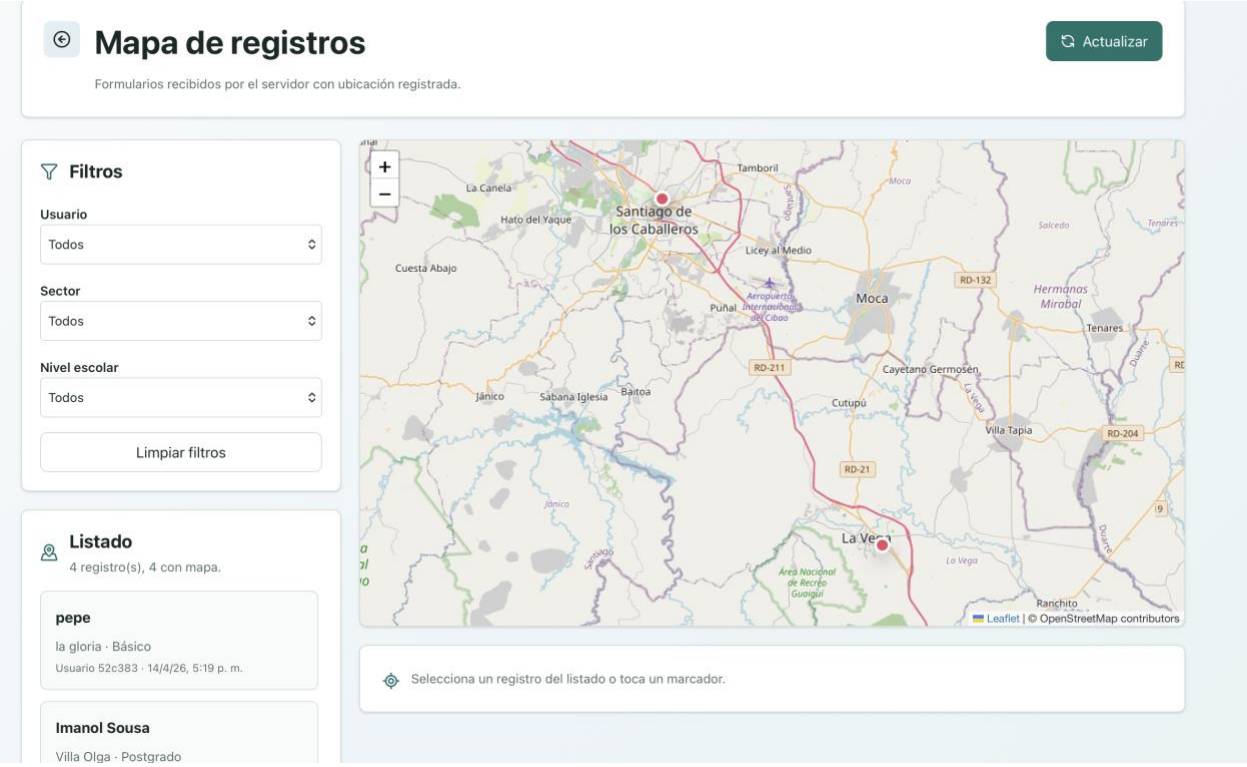
Cuando el servidor confirma que un formulario fue recibido y guardado, el worker actualiza el registro local como SYNCED y conserva el identificador generado por MongoDB. Si ocurre un error, el formulario se marca como ERROR y queda disponible para reintentar.



Mapa de Registros

La pagina de mapa consulta los formularios recibidos por el servidor y los presenta en un mapa interactivo usando Leaflet. Cada marcador representa un formulario con coordenadas validas. La pantalla tambien incluye filtros por usuario, sector y nivel escolar.

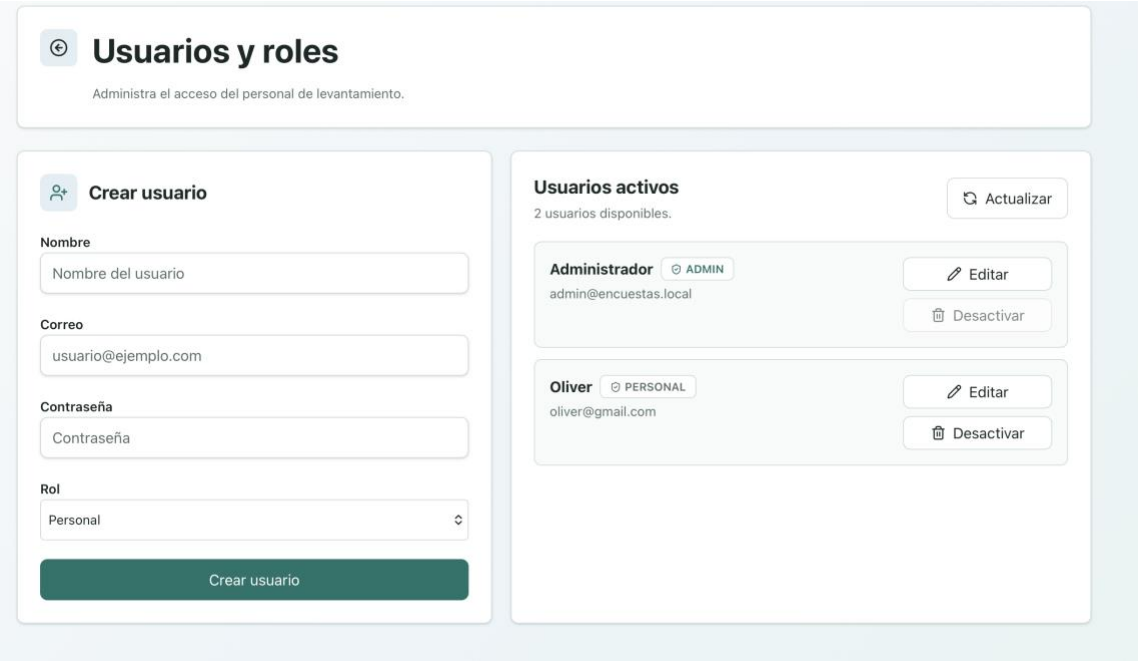
Al seleccionar un registro se muestra informacion adicional, incluyendo nombre del encuestado, sector, nivel escolar, usuario, fecha de recepcion y foto del registro cuando esta disponible.



Administración de Usuarios

La administracion de usuarios esta disponible para usuarios con rol ADMIN. Desde esta pantalla se pueden crear usuarios, modificar sus datos, asignar rol de administrador o personal, y desactivar usuarios.

Esta pantalla consume endpoints protegidos del backend, por lo que no puede ser utilizada por usuarios sin permisos administrativos.



Backend

El backend fue desarrollado en Java usando Javalin. Su funcion principal es exponer los servicios necesarios para autenticar usuarios, manejar formularios, administrar usuarios, sincronizar informacion y publicar operaciones mediante REST y gRPC.

La estructura del backend busca separar responsabilidades siguiendo una organizacion por capas:

- **Modelos:** representan las entidades principales del dominio.
- **DTOs:** definen los datos que entran y salen por la API.
- **Repositorios:** encapsulan el acceso a MongoDB.
- **Servicios:** contienen reglas de negocio y validaciones.
- **Utilidades:** manejan funciones transversales como JWT.
- **WebSocket:** atiende la sincronizacion de formularios desde el frontend.
- **gRPC:** publica operaciones para clientes externos.

Modelos del Dominio

Los modelos principales son Usuario y Formulario.

El modelo Usuario contiene:

- Identificador.
- Nombre.
- Correo.
- Hash de contrasena.
- Rol.
- Fecha de creacion.
- Estado activo o inactivo.

El modelo Formulario contiene:

- Identificador.
- Usuario que registro el formulario.
- Nombre del encuestado.
- Sector.
- Nivel escolar.
- Latitud.
- Longitud.
- Foto en Base64.
- Fecha de creacion.
- Estado de sincronizacion.
- Estado activo.

Tambien existe un enum para representar los niveles de educacion permitidos:

- Basico.
- Medio.
- Grado Universitario.
- Postgrado.
- Doctorado.

```java
@Entity("usuarios")
public class Usuario {
    @Id
    private ObjectId id;

    @Property("nombre")
    private String nombre;

    @Property("email")
    private String email;

    @Property("passwordHash")
    private String passwordHash;

    @Property("rol")
    private Rol rol = Rol.PERSONAL;

    @Property("fechaCreacion")
    private final LocalDateTime fechaCreacion = LocalDateTime.now();

    @Property("estado")
    private Boolean estado;

    public enum Rol{
        ADMIN, PERSONAL
    }
```

```java
public enum NivelEducacion {
    BASICO("BASICO"),
    MEDIO("MEDIO"),
    GRADO_UNIVERSITARIO("GRADO UNIVERSITARIO"),
    POSTGRADO("POSTGRADO"),
    DOCTORADO("DOCTORADO");

    private final String etiqueta;
    NivelEducacion(String etiqueta) {
        this.etiqueta = etiqueta;
    }

    public String getEtiqueta() {
        return etiqueta;
    }
}
```

```java
public class Formulario {

    @Id
    private ObjectId id;

    @Reference //para poder hacer referencia documento de usuario en la db
    private Usuario usuario;

    @Property("nombreEncuestado")
    private String nombreEncuestado;

    @Property("sector")
    private String sector;

    @Property("nivelEducacion")
    private NivelEducacion nivelEducacion;

    @Property("latitud")
    private double latitud;

    @Property("longitud")
    private double longitud;

    @Property("fotoBase64")
    private String fotoBase64;

    @Property("creadoEn")
    private final LocalDateTime creadoEn = LocalDateTime.now();

    @Property("sincronizado")
    private boolean sincronizado = false; //tiene valor false por defecto hasta confirmar con el cliente

    @Property("estado")
    private Boolean estado = true;
```

Servicios

Los servicios contienen la logica de negocio. El UsuarioService maneja la autenticacion, creacion de usuarios, actualizacion, listado, validaciones de datos y verificacion de permisos administrativos.

El FormularioService maneja la creacion de formularios, validacion de campos, verificacion del usuario registrador, actualizacion, eliminacion, listado y marcado de sincronizacion.

Centralizar estas reglas en servicios evita que las rutas HTTP tengan demasiada logica. Esto facilita el mantenimiento y permite que otros canales, como WebSocket o gRPC, puedan reutilizar o respetar reglas similares.

Repositorios y Persistencia

Los repositorios encapsulan el acceso a MongoDB. En lugar de consultar la base de datos directamente desde las rutas, el backend usa clases como UsuarioRepository y FormularioRepository.

MongoDB se utiliza como base de datos NoSQL. Morphia funciona como ODM, permitiendo mapear clases Java a colecciones de MongoDB. La configuracion de Morphia define la base de datos encuestas y el paquete donde estan las entidades del dominio.

Seguridad con JWT

La seguridad REST se basa en JWT. Cuando un usuario inicia sesion correctamente, el backend genera un token con el identificador del usuario, correo y rol. Ese token se envia en una cookie para que las solicitudes posteriores puedan ser autenticadas.

El filtro JWT valida la cookie de autenticacion, decodifica el token y coloca los datos del usuario en el contexto de la peticion. Para operaciones administrativas se valida adicionalmente que el rol sea ADMIN.

Este enfoque permite proteger los endpoints del backend y controlar el acceso segun el tipo de usuario.

WebSocket para Sincronizacion

El backend expone un WebSocket en /api/ws/sync. Este canal recibe formularios enviados desde el Web Worker del frontend.

El flujo general es:

1. El cliente abre una conexion WebSocket autenticada.
2. El servidor valida el token de la cookie.
3. El cliente envia un formulario pendiente en formato JSON.
4. El servidor convierte el mensaje a un DTO de formulario.
5. El servicio valida y guarda el formulario.
6. El servidor responde con confirmacion y el id generado.
7. El frontend marca el formulario local como sincronizado.

Este canal permite sincronizar registros pendientes sin depender de una llamada REST tradicional por cada formulario.

Servicio gRPC

Ademas del API REST, el backend publica un servicio gRPC. El contrato se define en el archivo .proto del proyecto. El servicio gRPC incluye dos operaciones principales:

- Crear un formulario.
- Listar formularios publicados por un usuario.

El servicio gRPC permite que clientes externos consuman operaciones del backend usando Protobuf y comunicacion gRPC. En este proyecto se implemento tambien un cliente grafico en Java Swing para demostrar el consumo del servicio.

Cliente gRPC

Es una aplicacion grafica hecha con Swing, donde se puede introducir el id del usuario, los datos del formulario, coordenadas y una imagen. La imagen se convierte a Base64 antes de enviarse al servidor gRPC.

Tambien permite listar formularios por usuario y mostrarlos en una tabla. Esto cumple con el requerimiento de crear un cliente elaborado para el servicio gRPC.

Base de Datos

MongoDB se utiliza como base de datos para almacenar usuarios y formularios cumpliendo con el requerimiento de utilizar una base de datos NoSQL, y permite guardar documentos con estructuras cercanas al modelo de la aplicacion.

Las colecciones principales son:

- usuarios
- formularios

Los usuarios almacenan informacion de acceso y rol. Los formularios almacenan la informacion de las encuestas, la referencia al usuario que registro el formulario, la ubicacion, foto en Base64 y estado de sincronizacion.

El uso de MongoDB Atlas permite que la base de datos se encuentre en la nube y pueda ser consumida por la aplicacion desplegada. El backend recibe la cadena de conexion mediante la variable de entorno MONGO_URI, evitando colocar credenciales directamente en el codigo fuente.

encuestas +

Cluster0 > encuestas

View monitoring | Visualize your data | + Create collection | Refresh

Collection name	Properties	Storage size	Data size	Documents	Avg. document size	Indexes	Total index size
formularios	-	425.98 kB	200.43 kB	5	40.09 kB	1	36.86 kB
usuarios	-	36.86 kB	437.00 B	2	218.00 B	1	36.86 kB

IP Access List

+ADD IP ADDRESS

You will only be able to connect to your cluster from the following list of IP Addresses:

IP Address	Comment	Status	Actions
0.0.0.0/0 (includes your current IP address)	access for all its	Active	EDIT DELETE
179.53.27.195/32	Created as part of the Auto Setup process	Active	EDIT DELETE

Docker y Despliegue

El proyecto incluye contenedores para facilitar el arranque del sistema. Existe un Dockerfile para el backend y otro para el frontend.

El backend se construye usando Gradle y se ejecuta como aplicacion Java. Expone el puerto 7070 para REST/WebSocket y el puerto 9090 para gRPC.

El frontend se construye con Node y Vite, luego se sirve mediante Nginx. La configuracion de Nginx tambien redirige las solicitudes /api hacia el backend, incluyendo soporte para WebSocket.

Docker Compose permite levantar ambos servicios juntos. El archivo docker-compose.yml define los servicios backend y frontend, las variables de entorno necesarias y los puertos expuestos.

Conclusión

El proyecto implementa una solucion web completa para la captura y sincronización de encuesta integrando tecnologías del lado del cliente con un backend organizado en capas, una base de datos NoSQL en MongoDB y servicios de comunicacion REST, WebSocket y gRPC.

La division entre frontend y backend permite mantener una arquitectura ordenada donde el frontend se encarga de la experiencia del usuario, almacenamiento local, captura de foto, geolocalizacion, visualizacion de mapa y sincronizacion desde el navegador. Asimismo, el backend se encarga de autenticacion, autorizacion, reglas de negocio, persistencia, WebSocket y gRPC.

El uso de React y Tailwind CSS, aunque no era obligatorio, aporto comodidad y claridad al desarrollo por la experiencia previa de Miguel Rodriguez con estas tecnologias hizo que el frontend fuera mas sencillo de construir, mantener y adaptar a dispositivos moviles.

Finalmente, el proyecto demuestra el uso de funcionalidades avanzadas de HTML5 y servicios modernos para resolver un caso practico donde la conectividad puede ser limitada. La capacidad de guardar formularios localmente y sincronizarlos despues es uno de los puntos mas importantes de la solucion, ya que responde directamente a la necesidad del levantamiento de datos en campo.

Antes de probar

- .env para ruta raíz (backend):

```
-   MONGO_URI=mongodb+srv://mgrg0001_db_user:UgIaJQ55KkG2aI4Y@cluster0.0q2mI2s.mongodb
    .net/encuestas?retryWrites=true&w=majority&appName=Cluster0
-
-   JWT_SECRET=6afcb4229b9e87a2552bcb009d64d8000bd86a3405f2e9033d01072ece868a15
```

- .env para carpeta frontend:

```
-   VITE_API_BASE_URL=/api
-
```

- Para correr de manera individual (sin docker) primero en la carpeta raiz se ejecuta *sh gradlew run* y en /frontend *npm run dev*.